

# Password Manager Using Java

VIT University

Second Year Java Project

Project Report

A console-based Java application for managing account records and demonstrating object-oriented programming concepts.

## 2. Introduction

Managing different usernames and passwords can become difficult when a user has several online accounts. People may also choose weak passwords or forget which account a particular password belongs to.

The purpose of this project is to build a simple password manager using Java and object-oriented programming concepts. The application provides a menu through which users can add web accounts and financial accounts, view stored accounts, update passwords, delete accounts, and save masked account records in a file.

The project also demonstrates practical use of Java concepts such as classes and objects, inheritance, abstraction, interfaces, method overloading, exception handling, ArrayList, file handling, and multithreading.

## 3. Problem Statement

Users often manage many account credentials, but keeping them organized manually can be inconvenient. A small application can make this process easier by keeping account information together and providing basic password-strength checks.

The problem addressed in this project is to create a Java-based password manager that can handle different types of accounts, validate passwords, prevent duplicate account IDs, allow changes to stored accounts, and save records without writing the complete password to the file.

The project is intended as an educational implementation of these ideas rather than a production password-security system.

## 4. Functional Requirements

The application provides a simple command-line menu. The user can add a Web Account by entering an ID, title, username, password, and URL. A Financial Account can also be added using an account number instead of a URL.

When an account is added, the password is checked according to the rules implemented in the corresponding account class. Passwords that are too short are rejected using a custom WeakPasswordException.

The application can display all current accounts with masked passwords. It can update an existing password, delete an account using its ID, and show the total number of account objects created. When the user chooses the save option, account information is appended to vault\_records.txt with the password masked.

## 5. Non-Functional Requirements

The program should be easy to run from a command line and simple enough for a user to understand without a graphical interface. The menu and messages are kept clear so that the user can follow each operation step by step.

The program should also be reasonably reliable. Invalid menu choices are handled, duplicate IDs are checked, weak passwords are rejected, and file-writing errors are caught instead of stopping the

application unexpectedly.

For basic security, the complete password is not displayed when account details are shown and the saved file contains a masked password. The project is not intended to provide encryption or the full security features expected from a real password manager.

## 6. System Architecture

The application follows a simple object-oriented structure. PasswordManagerApp is the main class and handles user input and the menu. VaultManager manages the ArrayList of account objects and provides operations such as adding, finding, updating, and deleting accounts.

VaultItem is the abstract parent class containing common account information such as ID, title, username, and password. WebAccount and FinancialAccount extend VaultItem and provide their own password-security rules and display methods.

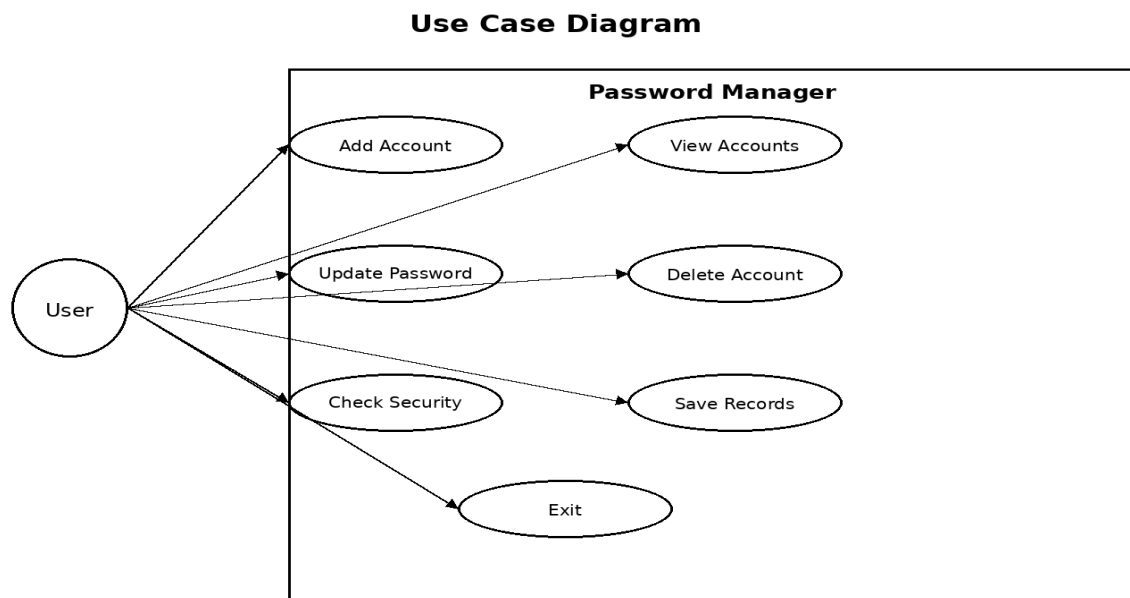
VaultSecurable defines the main operations related to security evaluation and saving. SecurityLevel is an enum used to represent WEAK, MODERATE, and STRONG results. VaultAutoLockService implements Runnable and runs the basic auto-lock timer in a separate thread.

This structure keeps the main program from having to contain all account-related logic in one place.

## 7. Design Diagrams

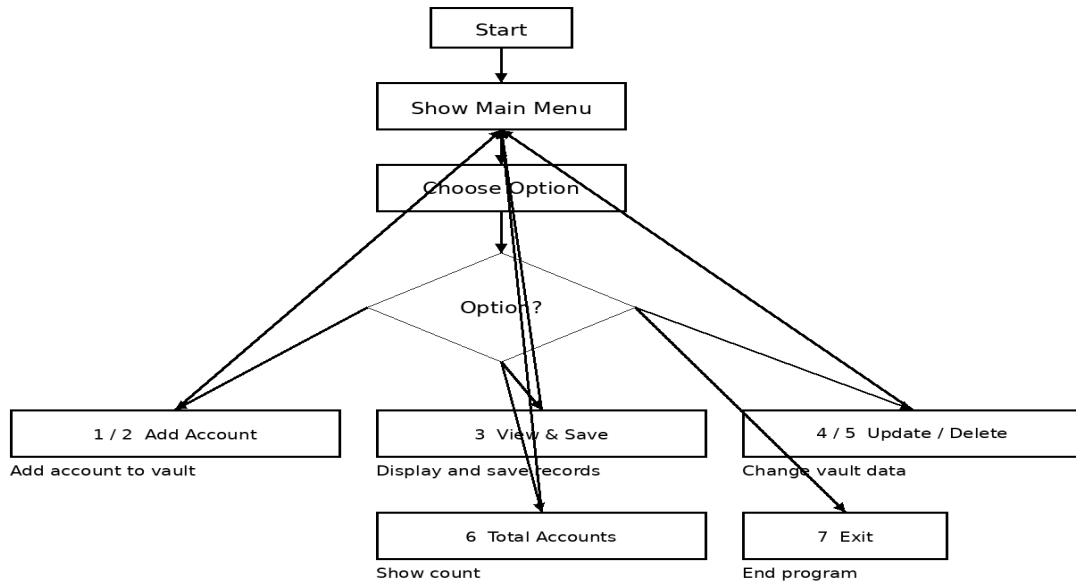
The diagrams below show the main parts and flow of the application. They are intentionally kept simple so that the design can be understood and explained easily during a project presentation.

### Use Case Diagram



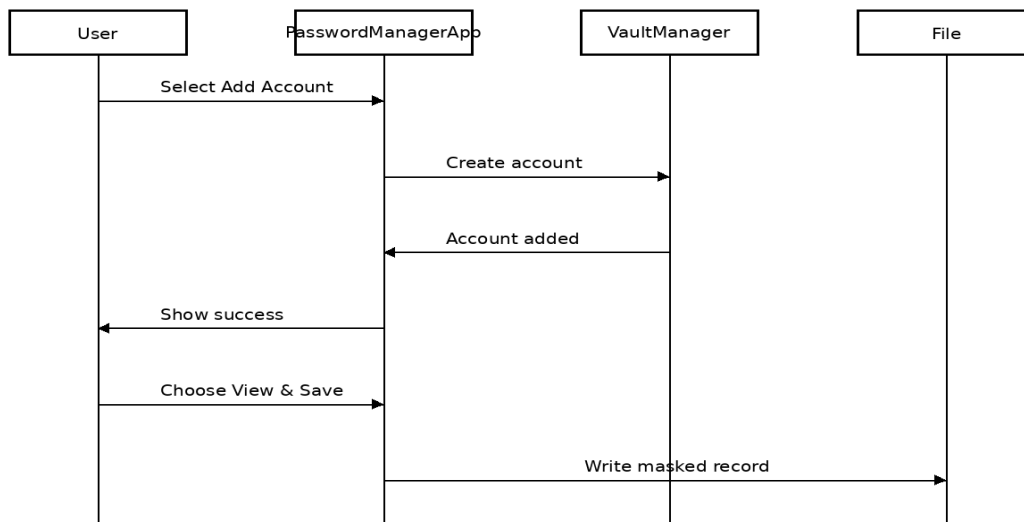
### Workflow Diagram

## Workflow Diagram



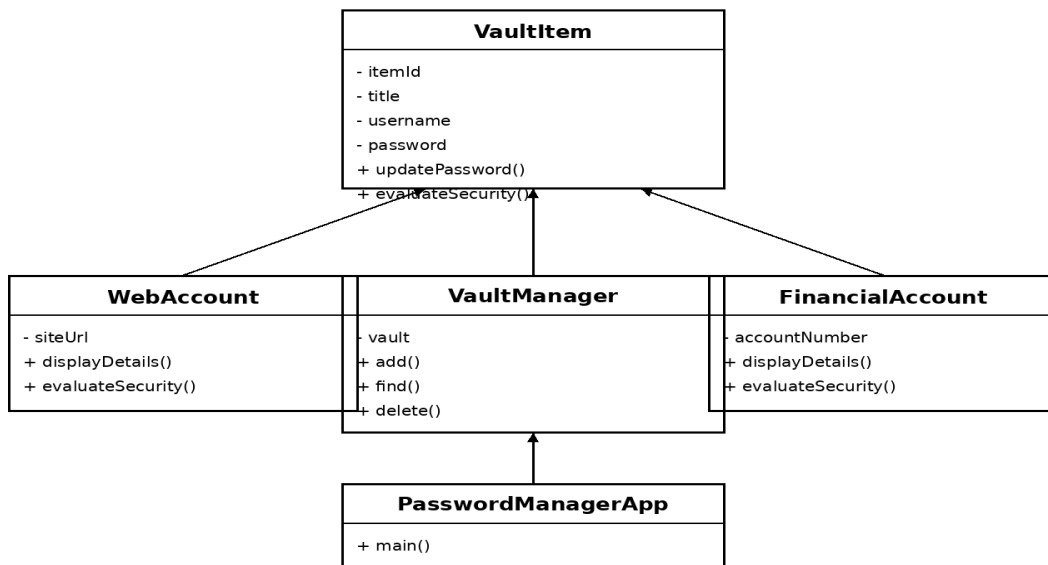
## Sequence Diagram

### Sequence Diagram - Add Account and Save



## Class Diagram

## Class Diagram



## 8. Design Decisions & Rationale

The project uses inheritance because **WebAccount** and **FinancialAccount** share common fields and operations. Keeping these common parts in **VaultItem** avoids writing the same code twice.

An interface, **VaultSecurable**, is used to define operations that account objects should support. The **SecurityLevel** enum makes the password-rating result easier to understand than using separate numbers or strings throughout the program.

**ArrayList** is used because the number of accounts is not fixed when the program starts. A custom **WeakPasswordException** is used so that an invalid password can be handled clearly in the main menu.

Passwords are masked while displaying or saving records. This is a basic security measure suitable for the scope of this student project. File handling uses a text file so that the project remains simple and easy to demonstrate.

## 9. Implementation Details

The program is divided into separate Java files so that each class has a clear responsibility. **VaultItem** stores the common account information and contains methods for displaying a masked password and saving a record. **WebAccount** and **FinancialAccount** override the security evaluation and display methods.

**VaultManager** maintains the list of **VaultItem** objects. It checks whether an ID already exists, finds an account by ID, updates passwords, and deletes accounts. **PasswordManagerApp** takes input from the user and calls these manager methods through the menu.

The password-strength rules are different for the two account types. Web accounts are considered strong when the password has at least 10 characters, a number, and a supported special character. Financial accounts require at least 12 characters, a number, and a supported special character for the strong level.

The application also uses method overloading for password updates and a Runnable-based timer for the auto-lock message.

## 10. Screenshots / Results

The application was tested through the command-line menu. Test cases covered adding web and financial accounts, checking moderate and strong passwords, handling duplicate IDs, finding an account, deleting an account, and confirming that a deleted account is no longer present.

In addition to the normal functional tests, 1,000 randomly generated password strings were tested against the password-validation logic. The tests completed successfully without compilation or runtime errors.

Screenshots of the running application can be added to this section after running the final version. The main results are that account operations work through the menu and saved records contain masked passwords rather than the full password.

## 11. Testing Approach

Testing was performed at two levels. First, individual functions and normal user operations were checked using a small Java test program. This included password evaluation for both account types, account addition, duplicate-ID checking, account searching, and deletion.

Second, random testing was used for the password rules. A set of 1,000 random password strings with different lengths and character combinations was passed through the validation logic. This helped check that the program handled different input patterns without crashing.

Invalid cases such as short passwords and duplicate IDs were also checked. Compilation was performed using javac and the project was run from the command line.

## 12. Challenges Faced

One challenge was deciding how to organize different account types without repeating the same code. This was handled by creating the VaultItem abstract class and extending it for WebAccount and FinancialAccount.

Another challenge was handling invalid passwords and file errors without making the program stop. A custom exception and try-catch blocks were used for this purpose.

Testing also required checking cases that are not part of the normal successful flow, such as duplicate IDs, deleted accounts, and randomly generated passwords. This helped identify whether the program behaved consistently with different inputs.

## 13. Learnings & Key Takeaways

This project gave practical experience with object-oriented programming in Java. The main concepts used were abstraction, inheritance, interfaces, polymorphism through overridden methods, method overloading, constructors, static members, enums, and exception handling.

The project also provided practice with ArrayList for storing objects, file handling for saving records, and Runnable and Thread for a simple background timer.

Another important learning was that testing is not limited to checking whether the program works once. Different input cases, invalid values, and repeated operations need to be checked to find problems before submission.

## **14. Future Enhancements**

The current project is intentionally simple and focuses on Java programming concepts. In a future version, passwords could be encrypted instead of only being masked when displayed or saved.

A database could be used instead of a text file so that records can be loaded again after restarting the program. A master password and proper authentication could also be added to protect the vault.

Other possible improvements include a graphical user interface, password generation, search and sorting options, stronger validation rules, and a more complete auto-lock mechanism that locks the vault after inactivity.

## **15. References**

1. Oracle Java Documentation – Java language and standard library concepts. 2. Java documentation for ArrayList, exception handling, file I/O, interfaces, inheritance, and threads. 3. VITyarthi project submission guidelines provided for the course.